

0.4. L'alphabet à la main, jusqu'au bout de la ligne

Une lettre par case : au bout de 20, la ligne est pleine.

1. Ce qu'il faut comprendre

Chaque lettre prend **une case de 8 pixels** de large. L'écran en fait 160 : **20 lettres** remplissent donc toute la ligne, de la colonne 0 à la colonne **19**.

On s'arrête à **T**, la 20e lettre. Il n'y a **pas de colonne 20** : `texte(20, 0, "U")` est refusé par le compilateur, qui répond que l'écran fait 20 colonnes.

Vingt lignes presque identiques : c'est long à écrire, et c'est exactement ce qu'une boucle saura faire à notre place.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **735 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         227 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         320 o ← CopierTuiles - 20 lettres sur 43
"A"            1 o ← texte() - un texte gravé
"B"            1 o ← texte() - un texte gravé
"C"            1 o ← texte() - un texte gravé
"D"            1 o ← texte() - un texte gravé
"E"            1 o ← texte() - un texte gravé
"F"            1 o ← texte() - un texte gravé
"G"            1 o ← texte() - un texte gravé
"H"            1 o ← texte() - un texte gravé
"I"            1 o ← texte() - un texte gravé
"J"            1 o ← texte() - un texte gravé
"K"            1 o ← texte() - un texte gravé
"L"            1 o ← texte() - un texte gravé
"M"            1 o ← texte() - un texte gravé
"N"            1 o ← texte() - un texte gravé
"O"            1 o ← texte() - un texte gravé
```

```
"P"          1 o ← texte() – un texte gravé
"Q"          1 o ← texte() – un texte gravé
"R"          1 o ← texte() – un texte gravé
"S"          1 o ← texte() – un texte gravé
"T"          1 o ← texte() – un texte gravé
```

Lettres gravées : ABCDEFGHIJKLMNOPQRST

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 20    220 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte>    // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {          // Le programme commence ici.
    // Une lettre par case, de gauche à droite, sur la ligne 0.
    // texte(colonne, ligne, "LETTRE") : seule la colonne change, de 1 en 1.
    texte(0, 0, "A");    // colonne 0 : la première à gauche
    texte(1, 0, "B");    // colonne 1 : juste à droite du A
    texte(2, 0, "C");
    texte(3, 0, "D");
    texte(4, 0, "E");
    texte(5, 0, "F");
    texte(6, 0, "G");
    texte(7, 0, "H");
    texte(8, 0, "I");
    texte(9, 0, "J");
    texte(10, 0, "K");    // colonne 10 : le milieu de la ligne
    texte(11, 0, "L");
    texte(12, 0, "M");
    texte(13, 0, "N");
    texte(14, 0, "O");
    texte(15, 0, "P");
    texte(16, 0, "Q");
    texte(17, 0, "R");
    texte(18, 0, "S");
    texte(19, 0, "T");    // colonne 19 : la dernière de la ligne
    // On s'arrête ici : l'écran fait 20 colonnes (de 0 à 19).
    // 20 lettres de 8 pixels = 160 pixels : toute la largeur de l'écran.
    // (si on dépasse : texte(20, 0, "U") est refusé par le compilateur,
    //  qui répond « l'écran fait 20 colonnes et 18 lignes ; 20,0 est dehors »)

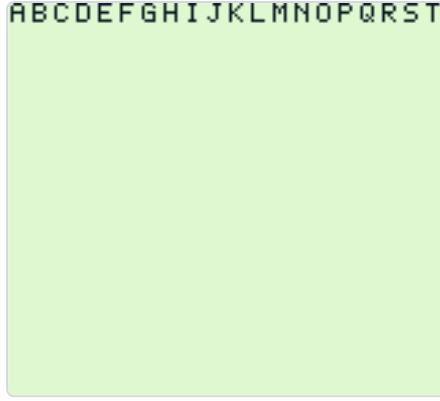
    while (true) {      // La boucle de jeu : recommence pour toujours...
        image();        // ... attendre l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur toute la première ligne, d'un bord à l'autre de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la ligne 0 est pleine : A à T
- ✓ rien ne déborde sur la ligne 1

Ensuite : **0.4.1. L'alphabet à la main, jusqu'au bout de la ligne — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++